

Telemetry Hardware Architecture: Direct Atomic Array

This document outlines the microarchitectural mechanics of the `Telemetry<Metric>` class, focusing on cache-coherency, pipeline throughput, and hardware-level interaction.

1. Hardware Memory Layout

The system uses static allocation to avoid heap fragmentation and uses `alignas(64)` to ensure that each thread's memory is isolated to a specific 64-byte hardware cache line.

Physical Memory Allocation (L3 / RAM View)

Offset	Memory Block	Hardware Ownership
0x...0000	ThreadStorage[0]	Core 0 (L1 Cache Line #0)
0x...0040	ThreadStorage[1]	Core 1 (L1 Cache Line #1)
0x...0080	ThreadStorage[2]	Core 2 (L1 Cache Line #2)

Because `ThreadStorage` is padded to 64 bytes, every index `i` is guaranteed to reside on a distinct cache line, preventing **False Sharing**.

2. Microarchitecture Mechanics (The Hot Path)

When a worker thread executes `count()` or `add()`, the CPU performs the following steps:

- Slot Lookup (TLS):** The thread accesses `slot_idx` via the Thread Local Storage segment register (e.g., `fs:[...]` on x86-64). This is a local register operation—zero bus traffic.
- L1 Cache Access:** The CPU checks for the target address in the private L1 Data Cache (L1d). Because we operate on a thread-local index, the line is already in the `Modified (M)` or `Exclusive (E)` state.
- Atomic Execution:** The `lock add` or `compare_exchange` instruction is executed.
 - Relaxed Ordering:** `std::memory_order_relaxed` ensures no pipeline stalls, no fence instructions (MFENCE/SFENCE), and no store-buffer draining.
 - MESI Protocol:** Since each thread owns its cache line, the CPU completes the atomic write without broadcasting invalidations to other cores.

3. Hardware Realities & Edge Cases

Bottleneck	How Telemetry Handles It	Impact
Mutex Locks	Lock-free design	Eliminated (0 OS wait state transitions).
L1 False Sharing	<code>alignas(64) padding</code>	Eliminated (No cache line bouncing).
Memory Barriers	<code>memory_order_relaxed</code>	Zero pipeline flushes; maximum IPC.
L2 Spatial Traffic	L2 Interconnect Prefetching	Minor L2 bus noise, undetectable by L1 pipeline.

The `collect_all()` Impact

When the reporter thread aggregates data, it forces the MESI protocol to transition worker-owned cache lines from **Modified (M)** to **Shared (S)**.

- **Cost:** Approximately 2–5 nanoseconds of micro-latency per cache line on the next worker write.
- **Verdict:** This is negligible, as reporting occurs at an orders-of-magnitude lower frequency than metric recording.

4. Hardware Optimization Checklist

- ☒ **No Mutex Stalls:** The worker never sleeps or waits.
- ☒ **L1 Cache Locality:** Metrics reside in the core's private, fastest memory.
- ☒ **Zero Inter-Core Contention:** Different cores never write to the same cache line.
- ☒ **Maximum Throughput:** Relaxed atomics allow the CPU to reorder instructions for maximum IPC (Instructions Per Cycle).

System Engineering Note: By design, this architecture is "Heisenbug-Free." The act of recording telemetry does not modify the timing of the hot-path kernel, ensuring that performance observations are accurate and reproducible.